Step 1: Setting up the OpenAI API

First, you'll need your OpenAI API key. For security, it's best to keep this in an environment variable. Here's how you can initialise the API client in Python:

```
import os
import openai
openai.api_key = os.getenv("OPENAI_API_KEY")
```

Step 2: Constructing the prompt

The prompt is the instruction you provide to the model. The quality and clarity of your prompt largely determine the output. For Java modernisation, prompts can vary depending on your goals—for example, general upgrades to newer Java syntax, or specific migration to Spring Boot.

Here's an example of a prompt for upgrading Java code to Java 17:

```
prompt = "
You are an expert Java developer. Modernize the following
Java code to conform to Java 17 standards, improving
readability, performance, and using new language features
where applicable. Please return only the updated Java code.
"
```

For a Spring Boot migration, you may use a prompt like:

```
prompt = "

Act as a senior Java developer. Refactor the following
servlet-based Java code to use Spring Boot, applying best
practices and using annotations. Return the modernized Spring
Boot code only.
"
```

Step 3: Calling the OpenAI API

Now, let's see how to make the API call and retrieve the response:

```
response = openai.ChatCompletion.create(
model="gpt-3.5-turbo",  # or another model, e.g., gpt-4
messages=[
{"role": "system", "content": "You are an expert Java
developer."},
{"role": "user", "content": prompt}
],
max_tokens=800
)
modernized_code = response["choices"][0]["message"]
["content"].strip()
print(modernized_code)
```

This code sends your prompt to the API and prints out the updated Java code.

Step 4: Building a full script

Let's put it all together. Below is a simple but complete Python script that reads Java code from a file, constructs a prompt, calls OpenAI, and writes the modernised code to a new file.

```
import os
import openai
def read_java_file(file_path):
with open(file_path, 'r') as f:
return f.read()
def write_java_file(file_path, content):
with open(file_path, 'w') as f:
f.write(content)
def modernize_java_code(legacy_code, task="upgrade"):
if task == "springboot":
prompt = f"""
Act as a senior Java developer. Refactor the following
servlet-based Java code to use Spring Boot, applying best
practices and using annotations. Return the modernised Spring
Boot code only.
```

Servlet-based Java code is:

```
{legacy_code}
"""
else:
prompt = f"""
You are an expert Java developer. Modernise the following
Java code to conform to the latest Java standards (e.g.,
Java 17), improving readability, performance, and using new
language features where applicable. Please return only the
updated code.
```

Legacy Java code is:

```
{legacy_code}
"""
response = openai.ChatCompletion.create(
model="gpt-3.5-turbo",
messages=[
{"role": "system", "content": "You are an expert Java
developer."},
{"role": "user", "content": prompt}
],
max_tokens=800
)
return response["choices"][0]["message"]["content"].strip()
if __name__ == "__main__":
openai.api_key = os.getenv("OPENAI_API_KEY")
input_file = "legacy_code.java"
output_file = "modernized_code.java"
```